Dhaka International University

Lab Report details:

Lab Report Topic : **Exception Handling in Java**
Lab Report No : 07
Course Title : Object-Oriented Programming Languages
Course Code : 0613-202

Submitted By:	Submitted To:
S M Nabil Ausaf Roll: 01 Reg No: CS-D-91-23-124492	Prof. Md. Abdul Based

Date of Submission: 15 Nov 2024

Title: Exception Handling in Java

Code:01

```
public class NewClass {
   public static void main(String[] args) {
int a, b;
      double c;
       try {
a = 10;
b = 5;
         c = a / b;
         System.out.println(c);
      } catch (ArithmeticException e) {
         System.out.println(e);
      }

      System.out.println("Successful");
   } }
```

Objective:

To comprehend and implement exception handling in Java using a try-catch block to manage arithmetic exceptions. The goal is to keep the program from terminating because of runtime errors and execute continuously without interruption.

Components Used:

1. Java Development Kit (JDK)
2. Integrated Development Environment (IDE) like IntelliJ IDEA, Eclipse, or NetBeans
3. Computer System with Java installed **Theory:**

Exception handling is a Java mechanism for managing runtime exceptions in a program to keep the normal flow of the program running. The **try** block would be utilized for enclosing the code that might throw an exception, and the **catch** block would be utilized for dealing with the exception if it is thrown. The **ArithmeticException** is an exception that is thrown when an arithmetic exception is raised, for example, division by zero.

If an exception occurs, control is abruptly transferred from the **try** block to the appropriate **catch** block. The exception object holds information about the error, which may be printed to facilitate

debugging. Exception handling enables programs to be more robust by not letting them crash and by providing simple recovery from errors.

Design Procedure:

1. Declare and initialize variables ‘a’, ‘b’, and ‘c’.
2. Perform division in a ‘try’ block and print the result.
3. Catch the **‘ArithmeticException’** which may be raised and print the error message.
4. Print a success message to denote successful execution of the program.
5. If ‘b’ is initialized to zero, the exception will be raised, and the catch block is executed.
6. Else, the result of division is printed, followed by the success message.

Code:

Main.java | NewClass.java | Exception_Handeling.java

```java
public class NewClass {
    public static void main(String[] args) {
        int a, b;
        double c;

        try {
            a = 10;
            b = 5;
            c = a / b;
            System.out.println(c);
        } catch (ArithmeticException e) {
            System.out.println(e);
        }

        System.out.println("Successful");
    }
}
```

Input and Output:

Input:

```
a = 10
b = 5
```

Output:

```
Result: 2.0
Successful
```

Input (When b=0):

```
Result: 2.0
Successful
```

Output:

```
Exception caught: java.lang.ArithmeticException: / by zero
Successful
```

Discussion:

1. The program runs without throwing any exception since ‘b’ is 5 and not zero.
2. On assigning a value of zero to ‘b’, an **‘ArithmeticException’** is thrown on division by zero, and the catch block deals with it gracefully.
3. Exception handling keeps the program from abruptly crashing, enhancing robustness and stability.
4. The last print statement "Successful" is run irrespective of whether an exception is thrown or not, providing a good flow control.
5. Java does have an exception hierarchy, and **‘ArithmeticException’** is a subclass of **‘RuntimeException’**, an unchecked exception.
6. Effective utilization of exception handling mechanisms aids in debugging and keeping the code error-free and clean.

Conclusion:

This lab show the application of exception handling in Java. The try-catch mechanism efficiently deals with arithmetic exceptions for uninterrupted program execution. Exception handling is an important concept to grasp for developing error-free and reliable Java applications. Proper exception handling enables developers to create applications that recover from errors gracefully, resulting in a good user experience and easier code maintenance.

Code: 02

```
public class Exception_Handeling {
   public static void main(String[] args) {
       try{
int a,b;
double c;
a=5;
b=0;
c=a/b;
         System.out.println(c);
```

```
    }
    catch(ArithmeticException e){//ArrayIndexOutOfBoundException
System.out.println(e);
    }
finally{
      System.out.println("successful");
    }
}
}
```

Objective:

To understand and implement exception handling in Java through the use of a try-catch-finally block with the aim of handling arithmetic exceptions and preventing interruptions.

Components Used:

1. Java Development Kit (JDK)
2. Integrated Development Environment (IDE) like IntelliJ IDEA, Eclipse, or NetBeans
3. Computer System with Java installed **Theory:**

Exception handling is an important feature of Java by which a program can handle runtime errors without ending in an abnormal way. The 'try' block holds code that may possibly throw an exception. The 'catch' block catches the particular exception and stops abrupt termination. The 'finally' block is run regardless of whether any exception is thrown or not and provides the execution of cleanup code if needed.

The '**ArithmeticException'** is thrown when an arithmetic operation, like division by zero, is tried. If unhandled, it will make the program abruptly end.

Design Procedure:

1. Initialize and declare variables 'a', 'b', and 'c'.

2. Implement a 'try' block to carry out division and try printing the result.

3. Catch any '**ArithmeticException'** thrown and print the error message.

4. Utilize a 'finally' block to print a success message irrespective of exception occurrence.

Code:

Main.java NewClass.java Exception_Handeling.java

```java
public class Exception_Handeling {
    public static void main(String[] args) {

        try{
            int a,b;
            double c;
            a=5;
            b=0;
            c=a/b;
            System.out.println(c);
        }
        catch(ArithmeticException e){//ArrayIndexOutOfBoundException
            System.out.println(e);
        }
        finally{
            System.out.println("successful");
        }
}
}
```

Input and Output:

Input:

```
a = 5
b = 0
```

Output:

```
java.lang.ArithmeticException: / by zero
successful
```

Discussion:

1. The code tries to divide ‘a’ by ‘b’, with ‘b’ being zero, hence an **‘ArithmeticException’**.
2. The exception is caught by the **‘catch’** block so that the program does not crash.

3. The **‘finally’** block executes irrespective of whether or not an exception is thrown, so any cleanup or messages required are done.
4. Exception handling improves program stability by handling run-time errors in a controlled way.

Conclusion:

This lab illustrated the application of exception handling in Java. With the **try-catch-finally** mechanism, we can properly handle arithmetic exceptions and keep the program stable. The ‘finally’ block guarantees that important code is run regardless of whether an exception is thrown, thereby promoting good programming habits.